

<pre> 1.import cv2 import numpy as np def detect_shapes(p): img = cv2.imread(p) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) blur = cv2.GaussianBlur(gray, (5,5), 0) edges = cv2.Canny(blur, 50, 150) contours,_ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) for c in contours: approx = cv2.approxPolyDP(c, 0.04*cv2.arcLength(c, True), True) x,y,w,h = cv2.boundingRect(approx) if len(approx) == 3: s = "Triangle" elif len(approx) == 4: s = "Square" if 0.9 <= w/h <= 1.1 else "Rectangle" elif len(approx) > 4: s = "Circle" else: s = "Unknown" cv2.drawContours(img, [approx], -1, (0,255,0), 2) cv2.putText(img, s, (x,y-10), 0, 0.5, (0,0,255), 2) cv2.imshow("Detected Shapes", img) cv2.waitKey(0) cv2.destroyAllWindows() detect_shapes('surveillance_image.jpg') </pre>	<pre> 2.import numpy as np import cv2 from matplotlib import pyplot as plt img = cv2.imread('bear.png') img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) dst = cv2.fastNlMeansDenoisin gColored(img, None, 10, 10, 7, 15) plt.subplot(121) plt.imshow(img) plt.title('Original') plt.subplot(122) plt.imshow(dst) plt.title('Denoised') plt.show() </pre>	<pre> 3.import cv2 import numpy as np image = cv2.imread('GeekforGeeks.jpg') gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) gray = np.float32(gray) dest = cv2.cornerHarris(gray, 2, 5, 0.07) dest = cv2.dilate(dest, None) threshold = 0.01 * dest.max() image[dest > threshold] = [0, 0, 255] corners = np.sum(dest > threshold) print("Number of corners detected:", corners) cv2.imwrite('corners_output.jpg' , image) cv2.imshow('Image with Borders', image) if cv2.waitKey(0) & 0xff == 27: cv2.destroyAllWindows() </pre>
---	---	---

<pre> 4.import cv2 import numpy as np import matplotlib.pyplot as plt img=cv2.imread('road_image.jpg') if img is None: exit() h,w=img.shape[:2] src=np.float32([[w*0.45,h*0.65],[w*0.55,h*0.65],[w*0.85,h*0.9],[w*0.15,h*0.9]]) dst=np.float32([[w*0.2,0],[w*0.8,0],[w*0.8,h],[w*0.2,h]]) M=cv2.getPerspectiveTransform(src,dst) bev=cv2.warpPerspective(img,M,(w,h)) edges=cv2.Canny(cv2.GaussianBlur(cv2.cvtColor(bev,cv2.COLOR_BGR2GRAY),(5,5),0),50,150) lines=cv2.HoughLinesP(edges,1,np.pi/180,50,minLineLength=50,maxLineGap=150) lane=np.zeros_like(bev) if lines is not None: for l in lines: x1,y1,x2,y2=l[0] cv2.line(lane,(x1,y1),(x2,y2),(0,255,0),5) out=cv2.addWeighted(bev,0.8,lane,1,1) plt.subplot(131); plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)); plt.axis('off') plt.subplot(132); plt.imshow(cv2.cvtColor(bev,cv2.COLOR_BGR2RGB)); plt.axis('off') plt.subplot(133); plt.imshow(cv2.cvtColor(out,cv2.COLOR_BGR2RGB)); plt.axis('off') plt.show() </pre>	<pre> 5. import cv2, numpy as np, mediapipe as mp mp_hands=mp.solutions.hands hands=mp_hands.Hands(0.7,0.7) draw=mp.solutions.drawing_utils cap=cv2.VideoCapture(0) def detect_corners(img): g=np.float32(cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)) c=cv2.dilate(cv2.cornerHarris(g,2,3,0.04),None) img[c>0.01*c.max()]=[0,0,255] return img def gesture(lm): t=lm[4].y<lm[3].y f=sum(lm[i].y<lm[i-2].y for i in [8,12,16,20]) return "Lights ON" if t and f==4 else "Lights OFF" if not t and f==0 else "Fan ON" if f==2 else "Fan OFF" if f==3 else "No Action" while cap.isOpened(): ret,frame=cap.read() if not ret: break frame=cv2.flip(frame,1) res=hands.process(cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)) if res.multi_hand_landmarks: for h in res.multi_hand_landmarks: draw.draw_landmarks(frame,h,mp_hands.HAND_CONNECTIONS) g=gesture(h.landmark) cv2.putText(frame,g,(10,50),0,1,(0,255,0),2) if g!="No Action": print(g) cv2.imshow("Gesture",detect_corners(frame)) if cv2.waitKey(1)&0xFF==ord('q'): break cap.release() cv2.destroyAllWindows() </pre>	<pre> 6. import cv2, torch, torchvision import numpy as np from torchvision import transforms import matplotlib.pyplot as plt model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True).eval() labels = ['__','person','bicycle','car','motorcycle','airplane','bus','train','truck'] def detect(img): t = transforms.ToTensor()(img).unsqueeze(0) with torch.no_grad(): p = model(t)[0] boxes = p['boxes'].cpu().numpy() labs = p['labels'].cpu().numpy() scores = p['scores'].cpu().numpy() res, names = [], [] for i in range(len(scores)): if scores[i] > 0.6 and labels[labs[i]] in ['car','bus','truck','motorcycle']: res.append(boxes[i]); names.append(labels[labs[i]]) return res, names img = cv2.imread('road_image.jpg') if img is None: exit() boxes, names = detect(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)) for i, b in enumerate(boxes): x1,y1,x2,y2 = map(int, b) cv2.rectangle(img,(x1,y1),(x2,y2),(0,255,0),2) cv2.putText(img,names[i],(x1,y1-5),0,0.7,(0,255,0),2) plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)) plt.title(f"Vehicles: {len(boxes)}") plt.axis('off') plt.show() cv2.imwrite("detected_vehicles.jpg",img) print("Vehicles detected:", len(boxes)) </pre>
---	---	---

<pre> 1. import nltk from nltk.tokenize import word_tokenize, sent_tokenize, regex_tokenize from nltk.stem import PorterStemmer import re try: nltk.data.find("tokenizers/punkt") except LookupError: nltk.download("punkt") stemmer = PorterStemmer() def preprocess_text(text): word_tokens = word_tokenize(text) sentence_tokens = sent_tokenize(text) regex_tokens = regex_tokenize(text, pattern=r"\s [\.,;\-!*]", gaps=True) whitespace_tokens = text.split() stemmed_tokens = [stemmer.stem(token) for token in word_tokens] return { "word_tokens": word_tokens, "sentence_tokens": sentence_tokens, "regex_tokens": regex_tokens, "whitespace_tokens": whitespace_tokens, "stemmed_tokens": stemmed_tokens } if __name__ == "__main__": text = "Running runs and ran are forms of the verb run. Here's another example: he was reading books!" processed_tokens = preprocess_text(text) print("Original Text:", text) print("\nTokenized by word_tokenize:", processed_tokens["word_tokens"]) print("\nTokenized by sent_tokenize:", processed_tokens["sentence_tokens"]) print("\nTokenized by regex_tokenize:", processed_tokens["regex_tokens"]) print("\nTokenized by whitespace split:", processed_tokens["whitespace_token s"]) print("\nStemmed Tokens:", processed_tokens["stemmed_tokens"]) </pre>	<pre> 2. from flask import Flask, request, jsonify, render_template from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.metrics.pairwise import cosine_similarity app = Flask(__name__) def are_questions_duplicate(q1, q2): vectorizer = TfidfVectorizer(ngram_range=(1, 2) vectors = vectorizer.fit_transform([q1, q2]) similarity = cosine_similarity(vectors[0], vectors[1])[0][0] print(f"Similarity score: {similarity}") return similarity > 0.5 # Lowered threshold from 0.7 to 0.5 @app.route('/') def home(): return render_template('index.html') @app.route('/predict', methods=['POST']) def predict(): try: data = request.json print("Received data:", data) # Debugging print statement if not data or "question1" not in data or "question2" not in data: return jsonify({"error": "Both questions are required"}), 400 q1 = data.get("question1", "").strip() q2 = data.get("question2", "").strip() if not q1 or not q2: return jsonify({"error": "Questions cannot be empty"}), 400 is_duplicate = are_questions_duplicate(q1, q2) return jsonify({"duplicate": bool(is_duplicate)}) except Exception as e: return jsonify({"error": str(e)}), 500 if __name__ == '__main__': app.run(debug=True) </pre>	<pre> 3. from flask import Flask, request, jsonify app = Flask(__name__) words_db = ["hello", "help", "hero", "world", "word", "welcome", "python", "programming", "predict", "project"] @app.route('/predict ', methods=['POST']) def predict(): text = request.form.get('inp ut_text', "").lower() last_word = text.split()[-1] if text else "" predictions = [w for w in words_db if w.startswith(last_wo rd)] return jsonify({"predictions" : predictions[:5]}) if __name__ == '__main__': app.run(debug=True) </pre>
---	---	---

<pre> 4. from flask import Flask, render_template, request import spacy, language_tool_python app = Flask(__name__) nlp = spacy.load('en_core_web_sm') tool = language_tool_python.LanguageTool('e n-US') def detect_tense_errors(doc): return ["Error: 'am' should be followed by 'going' instead of 'goes'." for token in doc if token.text.lower() == "am" for child in token.children if child.text.lower() == "goes"] @app.route('/', methods=['GET', 'POST']) def index(): sentence, parsed_output, grammar_errors, tense_errors = "", None, None, None if request.method == 'POST': sentence = request.form['sentence'] doc = nlp(sentence) noun_phrases = [c.text for c in doc.noun_chunks] or ["No noun phrases found."] verb_phrases = [t.text for t in doc if t.pos_ == "VERB"] or ["No verb phrases found."] tokens = [{"text": t.text, "pos": t.pos_} for t in doc] grammar_errors = [m.message for m in tool.check(sentence)] tense_errors = detect_tense_errors(doc) parsed_output = { "noun_phrases": noun_phrases, "verb_phrases": verb_phrases, "tokens": tokens } return render_template('index.html', sentence=sentence, parsed_output=parsed_output, grammar_errors=grammar_errors, tense_errors=tense_errors) if __name__ == '__main__': app.run(debug=True) </pre>	<pre> 5 . DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>NLP Text Analyzer</title> <link rel="stylesheet" href="style.css"> <link rel="stylesheet" href="https://cdnjs.cloudflare.c om/ajax/libs/font- awesome/5.15.3/css/all.min.css "> <link rel="icon" href="https://raw.githubusercontent.com/Redwiat/redwiat.git hub.io/main/favicon- redwiat.png"> </head> <body> <div class="container"> <h2>NLP Text Analyzer</h2> <form method="POST"> <textarea name="sentence" placeholder="Enter text" required></textarea> <button type="submit">Analyze</button> </form> {% if parsed_output %} <div class="output"> <h3>Noun Phrases:</h3> <p>{{ parsed_output.noun_phrases }}</p> <h3>Verb Phrases:</h3> <p>{{ parsed_output.verb_phrases }}</p> <h3>Tokens:</h3> {% for token in parsed_output.tokens %} {{ token.text }} {{ token.pos }} {% endfor %} <h3>Grammar Errors:</h3> <p>{{ grammar_errors }}</p> <h3>Tense Errors:</h3> <p>{{ tense_errors }}</p> </div> {% endif %} </div></body></html> </pre>	<pre> 6. from flask import Flask, render_template, request from textblob import TextBlob # For sentiment analysis app = Flask(__name__) # Define a function to perform sentiment analysis def analyze_sentiment(posts_ text): posts = posts_text.split('\n') # Split posts by new line results = [] for post in posts: if post.strip(): # Skip empty lines blob = TextBlob(post) sentiment_score = blob.sentiment.polarity sentiment = "Positive" if sentiment_score > 0 else "Negative" if sentiment_score < 0 else "Neutral" results.append((post, sentiment, sentiment_score)) return results @app.route('/', methods=['GET', 'POST']) def index(): results = None if request.method == 'POST': # Get the 'posts' data from the form try: posts_text = request.form['posts'] results = analyze_sentiment(posts_ text) except KeyError: pass # Handle the error if needed # Render the HTML template with the results return render_template('index.h tml', results=results) if __name__ == '__main__': app.run(debug=True) </pre>
--	---	--

<pre> 1. import numpy as np import tensorflow as tf # Data Preparation X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) y = np.array([0, 1, 1, 0]) # Neural Network Architecture model = tf.keras.Sequential() model.add(tf.keras.layers.Dense(2, input_dim=2, activation='relu')) model.add(tf.keras.layers.Dense(1, activation='sigmoid')) # Loss Function and Optimization model.compile(loss='binary_crossentropy', optimizer='adam') # Training model.fit(X, y, epochs=10000) # Testing predictions = model.predict(X) print(predictions) ----- 7. import numpy as np from tensorflow.keras.preprocessing.text import Tokenizer from tensorflow.keras.preprocessing.sequence import pad_sequences from tensorflow.keras import Sequential from tensorflow.keras.layers import Embedding, LSTM, Dense texts = ["I love this movie!", "This is great.", "I dislike it.", "This is terrible."] y = np.array([1,1,0,0]) tok = Tokenizer(1000); tok.fit_on_texts(texts) X = pad_sequences(tok.texts_to_sequences(texts), maxlen=20) test = ["This movie is fantastic!", "I can't stand this."] pred = model.predict(pad_sequences(tok.texts_to_sequences(test), maxlen=20)) for t,p in zip(test,pred): print(t,"->", "positive" if p>0.5 else "negative", f"({p[0]:.2f})") </pre>	<pre> 2. import tensorflow as tf from tensorflow.keras.datasets import cifar10 from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense from tensorflow.keras.utils import to_categorical (train_images, train_labels), (test_images, test_labels) = cifar10.load_data() train_images = train_images.astype('float32') / 255 test_images = test_images.astype('float32') / 255 train_labels = to_categorical(train_labels) test_labels = to_categorical(test_labels) model = Sequential() model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3))) model.add(MaxPooling2D((2, 2))) model.add(Conv2D(64, (3, 3), activation='relu')) model.add(MaxPooling2D((2, 2))) model.add(Conv2D(64, (3, 3), activation='relu')) model.add(Flatten()) model.add(Dense(64, activation='relu')) model.add(Dense(10, activation='softmax')) # 10 classes in CIFAR-10 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']) # Train the model model.fit(train_images, train_labels, epochs=5, batch_size=64) test_loss, test_accuracy = model.evaluate(test_images, test_labels) print(f'Test Accuracy: {test_accuracy * 100:.2f}%') </pre>	<pre> 3. import os, cv2, warnings, zipfile import numpy as np import pandas as pd import matplotlib.pyplot as plt import tensorflow as tf warnings.filterwarnings('ignore') # Extract dataset data_path = '/content/traffic-sign-dataset-classification.zip' with zipfile.ZipFile(data_path, 'r') as zip_ref: zip_ref.extractall('/content') # Load dataset path & labels dataset_path = '/content/traffic_Data/DATA' labels = pd.read_csv('labels.csv') # Display sample image img = cv2.imread(f'{dataset_path}/10/010_0011.png') img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) plt.imshow(img) plt.axis('off') plt.show() </pre>
--	--	--

<pre> 4. import tensorflow as tf import numpy as np import matplotlib.pyplot as plt img_shape = (28, 28, 1) # Generator def build_generator(): return tf.keras.Sequential([tf.keras.layers.Dense(256, input_dim=100), tf.keras.layers.LeakyReLU(0.2), # Discriminator def build_discriminator(): return tf.keras.Sequential([tf.keras.layers.Flatten(input_shape=img_shape), tf.keras.layers.Dense(512), tf.keras.layers.LeakyReLU(0.2), tf.keras.layers.Dense(1, activation='sigmoid')]) # Train GAN def train(epochs=10, batch_size=32): (X_train, _), _ = tf.keras.datasets.mnist.load_data() X_train = (X_train / 127.5 - 1).reshape(-1, 28, 28, 1) d_loss = 0.5 * (discriminator.train_on_batch(imgs, real) + discriminator.train_on_batch(generator_imgs, fake)) g_loss = gan.train_on_batch(noise, real) print(f'{epoch} [D loss: {d_loss[0]:.4f}] [G loss: {g_loss:.4f}]') discriminator.compile(loss='binary_crossentropy', optimizer=tf.keras.optimizers.Adam(0.0002, 0.5), metrics=['accuracy']) gan = tf.keras.Sequential([generator, discriminator]) gan.compile(loss='binary_crossentropy', optimizer=tf.keras.optimizers.Adam(0.0002, 0.5)) # Train train() </pre>	<pre> 5. import numpy as np import matplotlib.pyplot as plt from sklearn.preprocessing import MinMaxScaler from tensorflow.keras import Sequential from tensorflow.keras.layers import LSTM, Dense, Dropout # Generate data t = np.linspace(0, 100, 1000) data = np.sin(t) + np.random.normal(0, 0.1, t.shape) data[np.random.randint(200, 400, 10)] += np.random.normal(5, 2, 10) # Split split = int(0.8 * len(X)) X_train, X_test, y_train, y_test = X[:split], X[split:], y[:split], y[split:] # Model model = Sequential([LSTM(50, return_sequences=True, input_shape=(50,1)), Dropout(0.2), LSTM(50), Dropout(0.2), Dense(1)]) model.compile(optimizer='adam', loss='mse') # Train model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0) # Predict & detect anomalies y_pred = model.predict(X_test, verbose=0) error = np.abs(y_pred - y_test) threshold = np.percentile(error, 95) anomalies = error > threshold # Plot plt.plot(data, label='Data') plt.scatter(np.where(anomalies)[0]+split, y_pred[anomalies], c='red') plt.legend(); plt.show() print("Anomalies:", np.sum(anomalies)) </pre>	<pre> 6. import numpy as np, matplotlib.pyplot as plt from tensorflow.keras import layers, models from tensorflow.keras.datasets import mnist # Load & normalize data (x_train, _), (x_test, _) = mnist.load_data() x_train = x_train[...,:None]/255.0 x_test = x_test[...,:None]/255.0 # Add noise noise = lambda x: np.clip(x + 0.5*np.random.normal(size=x. shape), 0, 1) x_train_n, x_test_n = noise(x_train), noise(x_test) inp = layers.Input((28,28,1)) x = layers.Conv2D(32,3,activation='relu',padding='same')(inp) x = layers.Conv2D(32,3,activation='relu',padding='same')(x) x = layers.UpSampling2D(2)(x) out = layers.Conv2D(1,3,activation='sigmoid',padding='same')(x) model = models.Model(inp, out) model.compile(optimizer='adam', loss='binary_crossentropy') model.fit(x_train_n, x_train, epochs=5, batch_size=128, verbose=0) # Predict denoised = model.predict(x_test_n, verbose=0) # Show result plt.imshow(denoised[0].reshape(28,28), cmap='gray') plt.axis('off'); plt.show() </pre>
--	---	---

